

Taller · Sesión 1

AerolineaAPI

Repaso de conceptos clave · Spring Boot · IoC · DI · ORM · JPA · Hibernate

Este taller tiene dos partes. Primero un repaso de los conceptos centrales de la sesión — para que los tengas como referencia mientras construyes. Luego las tareas de **AerolineaAPI**, tu proyecto individual que aplica exactamente lo que acabas de ver.

SECCIÓN 1

Repaso de conceptos

El problema que Spring resuelve

En Java puro, cuando una clase necesita a otra, la crea con `new` adentro de sí misma. Eso funciona en proyectos pequeños, pero genera un problema llamado **acoplamiento**.

Java — sin Spring

```
public class ProductoService {  
    // ProductoService fabrica su propia dependencia  
    private ProductoRepository repo = new ProductoRepository();  
}
```

Consecuencias del `new` adentro:

Situación	Problema
Si <code>ProductoRepository</code> cambia su constructor	hay que cambiar <code>ProductoService</code> — y todo servicio que lo use
Si 10 servicios crean su propio repositorio	hay 10 instancias distintas; si debe ser una sola, está mal
Si quieren probar <code>ProductoService</code> sin BD real	no se puede — el repositorio está pegado adentro

- Acoplamiento: cuando una clase depende tan directamente de otra que cambiar una obliga a cambiar la otra.

Inversión de Control — IoC

DEFINICIÓN TÉCNICA

IoC (Inversion of Control) es un principio de diseño en el que el control de la creación y el ciclo de vida de los objetos se invierte: en lugar de que cada clase cree sus propias dependencias, esa responsabilidad se delega a un contenedor externo.

Spring Framework Documentation — Core Technologies, "Introduction to the Spring IoC Container"

Lo que se invierte es el control sobre el `new`. Con IoC, Spring crea los objetos y los entrega a quien los necesita.

Sin IoC

`ProductoService` hace `new ProductoRepository()` adentro

Con IoC

Spring hace el `new ProductoRepository()` y se lo entrega a `Pr`

El contenedor y los beans

DEFINICIÓN TÉCNICA

El contenedor IoC de Spring está representado por la interfaz `ApplicationContext`. Es el responsable de instanciar, configurar y ensamblar los objetos de la aplicación. A esos objetos que Spring administra se les llama beans.

Spring Framework Documentation — Core Technologies, "The IoC Container"

Al arrancar la aplicación, Spring crea todos los beans, los conecta entre sí, y levanta el servidor. El `main` solo le pasa el control — no instancia nada.

Inyección de Dependencias — DI

DEFINICIÓN TÉCNICA

La Inyección de Dependencias (Dependency Injection) es el mecanismo concreto con el que el contenedor IoC entrega las dependencias a los objetos que las necesitan. En lugar de que un objeto cree sus dependencias, estas le son inyectadas desde afuera — a través del constructor.

Spring Framework Documentation — Core Technologies, "Dependencies and Configuration in Detail"

IoC es el principio. DI es el mecanismo. El patrón que se usa en este módulo es siempre **constructor injection** con `@Autowired`:

```

@Service
public class VueloService {

    private final VueloRepository vueloRepository;

    @Autowired // Spring inyecta el repositorio aquí
    public VueloService(VueloRepository vueloRepository) {
        this.vueloRepository = vueloRepository;
    }
}

```

Elemento	Por qué
Campo final	La dependencia no puede cambiar después de ser inyectada
Sin new	Spring construye el repositorio — no ProductoService
@Autowired	Le dice a Spring: busca un bean de ese tipo y pásalo aquí

Las tres anotaciones de componente

Anotación	Capa	Responsabilidad
@Component	Cualquiera	Componente genérico — sin capa específica
@Service	Lógica de negocio	Procesa datos, toma decisiones
@Repository	Acceso a datos	Habla con la base de datos

- Si una clase que necesitas inyectar no tiene @Service, @Repository o @Component, Spring lanza NoSuchBeanDefinitionException al arrancar. Siempre marca las clases que Spring debe administrar.

ORM — Mapeo Objeto-Relacional

Java trabaja con objetos. Postgres trabaja con tablas. ORM es el puente que convierte unos en otros automáticamente.

DEFINICIÓN TÉCNICA

El Mapeo Objeto-Relacional (ORM) es una técnica que convierte datos entre el sistema de tipos de un lenguaje orientado a objetos y una base de datos relacional. Un framework ORM automatiza esa conversión: persiste objetos como filas y recupera filas como objetos, sin que el desarrollador escriba SQL manual para cada operación.

Martin Fowler — "Patterns of Enterprise Application Architecture", "Object-Relational Mapping"

Sin ORM — SQL manual

```
String sql = "INSERT INTO vuelo (origen, destino...) VALUES (?, ?, ...)";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, vuelo.getOrigen()); // un set por campo
ps.executeUpdate();
```

Con ORM — JPA

```
vueloRepository.save(vuelo);
```

JPA e Hibernate — no son lo mismo

	JPA	Hibernate
¿Qué es?	Especificación (contrato)	Implementación (quien hace el trabajo)
¿Qué define?	Qué operaciones deben existir y cómo comportarse	Cómo se ejecutan esas operaciones — genera SQL
¿Quién lo usa?	Spring Boot lo referencia	Spring Boot lo incluye por defecto

JPA — JAKARTA PERSISTENCE API

Especificación de Jakarta EE que define una API estándar para el mapeo objeto-relacional en Java. Es un contrato: define qué debe existir, no cómo implementarlo.

Jakarta Persistence 3.1 Specification — sección 1 "Introduction"

HIBERNATE ORM

Implementación de la especificación JPA. Es la biblioteca concreta que genera el SQL, administra las conexiones y hace funcionar las operaciones definidas por JPA. Spring Boot lo usa por defecto.

Hibernate ORM Documentation — "Introduction"

- Cuando escribes código JPA en este módulo (@Entity, @Id, JpaRepository), usas la API estándar. Hibernate es quien ejecuta el trabajo real por debajo.

Anotaciones JPA esenciales

Anotación	Dónde va	Qué hace
@Entity	Clase	Le dice a Hibernate: esta clase es una tabla
@Table(name = "...")	Clase	Nombre explícito de la tabla en la BD
@Id	Campo	Marca la clave primaria de la entidad
@GeneratedValue(IDENTITY)	Campo id	Postgres asigna el ID con autoincremento (BIGSERIAL)
@Column(nullable = false)	Campo	Genera restricción NOT NULL en la columna
@Enumerated(EnumType.STRING)	Campo enum	Guarda el nombre del enum como texto, no como número

Constructor vacío en entidades JPA

JPA requiere que toda entidad tenga un constructor sin argumentos. Hibernate lo usa internamente para crear instancias al leer filas de la base de datos. Si lo borran, JPA lanza una excepción al intentar leer datos.

Java

```
public class Vuelo {  
  
    public Vuelo() {}    // obligatorio – no borrar  
  
    public Vuelo(String origen, String destino, ...) { ... }  
}
```

Enum vs. clase para valores fijos

Cuando el conjunto de valores posibles es fijo y definido por el negocio, se usa **enum**. Una clase permite crear instancias con cualquier valor — el compilador no protege nada. Un enum garantiza que solo existan los valores declarados.

Con clase — sin protección

```
new EstadoVuelo("CANCELADO_HOY") // válido  
new EstadoVuelo("a_tiempo")      // válido  
new EstadoVuelo("")               // válido
```

Con enum — compilador protege

```
public enum EstadoVuelo {  
    PROGRAMADO,  
    EN_VUELO,  
    ATERRIZADO,  
    CANCELADO  
}
```

JAVA LANGUAGE SPECIFICATION SE 21 — SECCIÓN 8.9

Un tipo enum tiene un conjunto fijo y cerrado de instancias declaradas. El compilador garantiza que no es posible crear instancias adicionales en tiempo de ejecución.

JpaRepository — métodos disponibles sin escribir nada

Al extender JpaRepository<T, ID>, Spring Data JPA genera en tiempo de ejecución la implementación completa. **T** es la entidad, **ID** es el tipo de su @Id.

Método	SQL generado
findAll()	SELECT * FROM vuelo
findById(id)	SELECT * FROM vuelo WHERE id = ?
save(vuelo)	INSERT si no tiene ID · UPDATE si tiene ID
deleteById(id)	DELETE FROM vuelo WHERE id = ?
count()	SELECT COUNT(*) FROM vuelo
existsById(id)	SELECT CASE WHEN COUNT(id) > 0 ...

SECCIÓN 2

Taller — AerolineaAPI

Vas a construir **AerolineaAPI** aplicando exactamente lo que viste en clase. El dominio es distinto al de ShopAPI, pero la arquitectura es idéntica: mismos paquetes, misma convención de nombres, mismo patrón de inyección.

El sistema que vas a construir

AerolineaAPI es el backend de una aerolínea. Gestiona vuelos, pasajeros y reservas. A lo largo de las tres sesiones del módulo el sistema va a crecer progresivamente — cada sesión agrega una capa nueva sobre lo anterior.

Sesión	Qué se construye	Qué queda funcionando
S1 — hoy	Vuelo, Pasajero, sus enums, repositorios, servicios y endpoints REST básicos	Se obtienen los pasajeros desde la base de datos
S2	Reserva (vincula pasajero con vuelo), DTOs, endpoints POST y GET completos	Se crea una reserva enviando el id del pasajero y el id del vuelo
S3	Validaciones, manejo de errores, PUT, DELETE, documentación Swagger	API lista para producción: rechaza datos inválidos

Entidades del sistema completo

Así se ve el sistema cuando esté terminado. Hoy construyes las dos primeras entidades.

Entidad	Campos principales	Cuándo
Vuelo	id, origen, destino, fechaHora, estado (EstadoVuelo)	S1 — hoy
Pasajero	id, nombre, apellido, documento, email	S1 — hoy
Reserva	id, fechaReserva, clase (ClaseAsiento), pasajero, vuelo	S2

Enums del sistema

Enum	Valores	Cuándo
EstadoVuelo	PROGRAMADO · EN_VUELO · ATERRIZADO · CANCELADO	S1 — hoy
ClaseAsiento	ECONOMICA · EJECUTIVA · PRIMERA_CLASE	S2

- Hoy construyes: Vuelo, Pasajero, EstadoVuelo, sus repositorios, servicios y un GET básico. Reserva y ClaseAsiento aparecen en S2 cuando veamos cómo relacionar entidades.

Tareas

0
1

Preparar el proyecto

- Abre el proyecto de S0 que ya tienes en IntelliJ
- Agrega al pom.xml: spring-boot-starter-data-jpa y el driver de PostgreSQL
- Recarga Maven después de guardar — espera a que descargue las dependencias

0
2

Crear la base de datos

- Abre pgAdmin o psql
- Crea la base de datos: CREATE DATABASE aerolineaapi;
- Esta es la base vacía — Hibernate crea las tablas al arrancar

0
3

Configurar application.properties

- Define la URL de conexión apuntando a aerolineaapi
- Configura usuario y contraseña de tu Postgres local
- Agrega el dialecto de PostgreSQL
- Usa ddl-auto=update y activa show-sql=true

Recuerda el formato de la URL: jdbc:postgresql://localhost:5432/aerolineaapi

0
4

Crear la estructura de paquetes

- Crea los cuatro paquetes dentro del paquete raíz de tu proyecto
- model · repository · service · controller
- Cada paquete tiene una responsabilidad única — respétalos siempre

0
5

Crear el enum EstadoVuelo

- Crea EstadoVuelo en el paquete model
- Cuatro valores: PROGRAMADO, EN_VUELO, ATERRIZADO, CANCELADO
- Piensa antes de escribir: ¿por qué enum y no clase para representar el estado de un vuelo?

06

Crear la entidad Vuelo

- Anota la clase con `@Entity` y define el nombre de tabla explícitamente
- Campo id: tipo Long, con `@Id` y `@GeneratedValue(strategy = GenerationType.IDENTITY)`
- Campos origen y destino: String, `@Column(nullable = false)`
- Campo fechaHora: LocalDateTime, `@Column(nullable = false)`
- Campo estado: tipo EstadoVuelo, con `@Enumerated(EnumType.STRING)`
- Constructor vacío obligatorio para JPA + constructor con todos los campos
- Getters y setters para todos los campos

Recuerda: sin `@Enumerated(EnumType.STRING)`, Hibernate guarda números en la BD.

07

Crear la entidad Pasajero

- Misma estructura que Vuelo — `@Entity`, `@Table`, `@Id`, `@GeneratedValue`
- Campos: nombre, apellido, documento, email — todos String, `nullable = false`
- Constructor vacío + constructor completo + getters y setters

08

Crear los repositorios

- VueloRepository en el paquete repository — extiende JpaRepository
- PasajeroRepository en el paquete repository — extiende JpaRepository
- Ambos son interfaces, no clases. El cuerpo queda vacío por ahora.
- Anota cada uno con `@Repository`

09

Crear los servicios

- VueloService en el paquete service — anota con `@Service`
- Inyecta VueloRepository por constructor con `@Autowired` — campo final
- Métodos: `findAll()` y `save(Vuelo vuelo)`
- PasajeroService — misma estructura, inyecta PasajeroRepository

010

Crear los controllers y verificar

- VueloController en el paquete controller — `@RestController`, `@RequestMapping("/vuelos")`
- Inyecta VueloService por constructor
- Método: `@GetMapping` que devuelve List
- PasajeroController — misma estructura, ruta `/pasajeros`
- Corre la aplicación y verifica en consola que Hibernate crea las tablas
- Inserta un dato de prueba en pgAdmin e inspecciona la respuesta en Postman

Criterio de cierre

- ✓ Al terminar debes poder mostrar: (1) la tabla vuelo y la tabla pasajero creadas en pgAdmin con las columnas correctas, (2) una respuesta en Postman a GET /vuelos y GET /pasajeros — aunque sea un array vacío.

Errores frecuentes

Error	Causa probable	Solución
NoSuchBeanDefinitionException	Clase inyectada sin @Service, @Repository, @Component	Agregar la anotación correspondiente
Connection refused	Postgres no está corriendo o credenciales incorrectas	Verificar que Postgres esté activo; revisar credenciales
Las tablas no se crean	ddl-auto no está en update	Verificar spring.jpa.hibernate.ddl-auto=update
Columna estado guarda 0, 1, 2	Falta @Enumerated(EnumType.STRING)	Agregar la anotación; limpiar la tabla si ya tiene datos
No default constructor for entity	Falta el constructor vacío en la entidad	Agregar public Vuelo() {} y public Pasajero()